



Task Report

Secure Code Review



- **Prepared By:** Muhammad Izaz Haider
- **Assigned by:** Intern Intelligence
- **Role:** Penetration Tester
- **Date:** 7 April 2025

Table of Contents

#	Section Title	Page No.
i.	Cover Page	i
ii.	Table of Contents	ii
1.	Introduction	1
2.	Lab Objectives	1
3.	Background / Scenario	2
4.	Required Resources	2
5.	Vulnerable Code Overview	3/4/5
6.	Security Flaws Identified	6/7/8/9
7.	Recommendations	10/11
8.	Conclusion	12

1: Introduction

Secure code review is the process of analyzing an application's source code to identify potential security vulnerabilities before they can be exploited. This method ensures that the application follows best security practices, such as proper input validation, secure data handling, and robust authentication mechanisms. In this lab, I reviewed a provided codebase to identify common vulnerabilities and security flaws. The goal was to detect issues like improper error handling, insecure data storage, and potential injection vulnerabilities, which could pose risks to the application's integrity. Based on the findings, I have suggested actionable recommendations to improve the application's security posture.

2. Lab Objectives

This lab aimed to achieve the following goals:

- Identify security vulnerabilities in the codebase
- Evaluate the potential risks of these vulnerabilities
- Provide recommendations for remediation

3. Background / Scenario

Secure code review plays a pivotal role in identifying vulnerabilities in an application before they can be exploited. By reviewing the codebase, security flaws such as improper input handling, poor authentication, and insecure data storage can be detected early. This proactive approach is essential to ensure that the application is secure and resilient against potential attacks. In this lab, I performed a detailed review of a provided codebase to assess its security and provide recommendations for improvement. The goal was to identify common coding flaws and ensure that the application adheres to best security practices.

4. Required Resources

- Internet Access
- Source Code to Review
- Note-taking Application
- Laptop/PC
- Integrated Development Environment (IDE) I used VsCode

5. Vulnerable Code Overview

In this section, I will provide an overview of the Python code I have reviewed. The code is designed to perform two main tasks: user login and file uploading. Here's a breakdown of its functionality and how it operates.

I started by creating a simple login system that connects to an SQLite database (users.db). This database contains a table (users) where user credentials, such as the username and password, are stored. The script allows users to input their username and password to authenticate themselves.

When users attempt to log in, the program performs an SQL query to match the provided credentials with those stored in the database. If the credentials are correct, the user is welcomed; otherwise, an error message is shown. The credentials for the user admin are hardcoded into the database for simplicity.

Additionally, I implemented a file upload system that allows users to upload files to the server. Users are prompted to input the filename of the file they want to upload. Once the file name is provided, the script reads the content of the file and saves it to a specific directory on the server called uploads/. The file is saved using its original filename, making it easy to access later.

The program runs in a continuous loop where users can select between the following options:

- **Login:** Users can enter their username and password to authenticate.

- **Upload File:** Users can upload a file to the server.
- **Exit:** The program exits.

5.1 ScreenShots of Code

```
codebase.py •
codebase.py > login
1  import os
2  import sqlite3
3
4
5  conn = sqlite3.connect('users.db')
6  cursor = conn.cursor()
7  cursor.execute('CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)')
8  cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin123')") # Hardcoded
9  conn.commit()
10
11 def login():
12     print("=== Login Page ===")
13     username = input("Enter username: ")
14     password = input("Enter password: ")
15
16
17     query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
18     cursor.execute(query)
19     result = cursor.fetchone()
20
21     if result:
22         print(f"Welcome, {username}!")
23     else:
24         print("Invalid credentials.")
25
```

```
codebase.py > login
26 def upload_file():
27     filename = input("Enter the file name to upload: ")
28
29
30
31     with open(filename, 'rb') as f:
32         content = f.read()
33
34     upload_path = f"uploads/{filename}"
35     with open(upload_path, 'wb') as f:
36         f.write(content)
37     print(f"File uploaded to {upload_path}")
38
39 def main():
40     while True:
41         print("\n1. Login\n2. Upload File\n3. Exit")
42         choice = input("Choose an option: ")
43         if choice == '1':
44             login()
45         elif choice == '2':
46             upload_file()
47         elif choice == '3':
48             break
49         else:
50             print("Invalid option.")
51
```

```
codebase.py > ...
38
39 def main():
40     while True:
41         print("\n1. Login\n2. Upload File\n3. Exit")
42         choice = input("Choose an option: ")
43         if choice == '1':
44             login()
45         elif choice == '2':
46             upload_file()
47         elif choice == '3':
48             break
49         else:
50             print("Invalid option.")
51
52 if __name__ == "__main__":
53     if not os.path.exists("uploads"):
54         os.mkdir("uploads")
55     main()
56
```

In summary, this Python code allows basic user authentication and file upload functionality. It uses an SQLite database to store credentials and provides users with the ability to upload files to a designated directory. This code is designed for educational purposes and provides a foundation for more complex applications that could benefit from enhancements in security and functionality.

6. Security Flaws Identified

While reviewing the code, I found multiple security issues that can be exploited by attackers. Below I have identified and explained the key vulnerabilities present in the script, along with the exact code lines where these flaws exist.

1. Hardcoded Credentials

- In the initial part of the code, I noticed that admin credentials are hardcoded directly into the database. This is a risky practice because anyone with access to the code can see the credentials and misuse them.

```
cursor.execute('CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)')
cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin123')")
conn.commit()
```

- Having static credentials reduces flexibility and increases the risk of unauthorized access if the code is leaked or compromised.

2. SQL Injection Vulnerability

- The login function uses string formatting to build an SQL query. This makes the application vulnerable to SQL injection attacks where a malicious user can input crafted values to bypass authentication or even access the entire database.

```
query = f"SELECT * FROM users WHERE username = '{username}' AND password = '{password}'"
cursor.execute(query)
result = cursor.fetchone()
```

- Here, no input validation or parameterized queries are used, allowing attackers to manipulate the SQL logic.

3. Unrestricted File Upload

- In the `upload_file()` function, the file input from the user is directly saved into the `uploads/` directory without checking the file type or restricting executable files.

```
print("\n=== File Upload ===")
filename = input("Enter the file name to upload: ")

...

with open(filename, 'rb') as f:
    ...
    content = f.read()
    ...

upload_path = f"uploads/{filename}"
with open(upload_path, 'wb') as f:
```

- This can be exploited by an attacker to upload malicious scripts or executables (e.g., `.exe`, `.php`, `.bat`) which might later be executed or opened by the system.

4. No Input Validation or Sanitization

- Throughout the code, I saw that user input (both username/password and file name) is accepted directly without any kind of validation or sanitization. This makes the application weak against a wide range of attacks such as path

traversal, command injection, or malformed input that could crash the app.

```
print("=== Login Page ===")
username = input("Enter username: ")
```

And 2nd one is

```
print("\n=== File Upload ===")
filename = input("Enter the file name to upload: ")
```

- If a user enters a filename like ../../etc/passwd, it may attempt to access system files if not handled properly.

5. Plaintext Password Storage

- Passwords are stored in the database in plaintext. This is a serious security issue, as anyone gaining access to the database can directly see all user passwords.

```
conn = sqlite3.connect('users.db')
cursor = conn.cursor()
cursor.execute('CREATE TABLE IF NOT EXISTS users (username TEXT, password TEXT)')
cursor.execute("INSERT INTO users (username, password) VALUES ('admin', 'admin123')")
conn.commit()
```

- Passwords should always be stored using hashing algorithms (e.g., bcrypt, SHA-256) with proper salting to ensure they are secure even if the database is compromised.

These identified flaws pose serious risks to the application's security. From hardcoded credentials that expose sensitive login info, to SQL injection vulnerabilities that can allow unauthorized database access, each issue opens doors for potential exploitation. Unrestricted file uploads and lack of input validation further widen the attack surface, while storing passwords in plaintext can completely compromise user privacy.

Such vulnerabilities can be actively exploited by attackers to bypass authentication, upload malicious files, steal user data, or gain system-level access. These issues collectively weaken the entire security posture of the application.

In the following section, I will share my recommendations to fix these flaws, improve coding practices, and ensure the application is more resilient against real-world attacks.

7. Recommendations

After carefully reviewing the code and identifying several vulnerabilities, I have come up with the following recommendations to improve the security of the application:

- **Avoid Hardcoded Credentials**

I recommend never storing credentials directly in the code. Instead, use secure environment variables or a configuration management system to store sensitive data like usernames and passwords.

- **Use Parameterized Queries (Prepared Statements)**

To prevent SQL injection, I suggest replacing the dynamic SQL query with parameterized queries. This ensures that user inputs are treated as data, not executable code. For example:

```
$ CURSOR.EXECUTE("SELECT * FROM USERS WHERE USERNAME = ?  
AND PASSWORD = ?", (USERNAME, PASSWORD))
```

- **Hash and Salt Passwords**

Storing plaintext passwords is extremely risky. Passwords should be hashed using secure algorithms like bcrypt or argon2, and salted to prevent rainbow table attacks.

- **Implement Input Validation**

Validate and sanitize all user inputs to avoid unexpected behaviors. This includes usernames, passwords, and especially file names during upload.

- **Secure File Upload Mechanism**

Allow only specific file types for upload, check file extensions, and consider scanning files for malware. Uploaded files should also be renamed and stored in a safe location to avoid overwriting or executing harmful content.

- **Use Role-Based Access Controls (RBAC)**

Add user roles and proper access control mechanisms to ensure that only authorized users can perform sensitive actions like uploading files.

- **Exception Handling and Logging**

Implement proper error handling to avoid exposing internal logic. Also, maintain secure logs of failed login attempts or suspicious uploads for monitoring purposes.

- **Directory Traversal Protection**

Sanitize file names and avoid using raw user input to construct file paths. This prevents attackers from accessing unauthorized directories or system files.

By following these recommendations, the security of the application can be significantly improved, making it more resilient to real-world threats and attacks. These practices are considered standard in secure software development and can greatly reduce the chances of exploitation.

8. Conclusion

This secure code review helped me understand how insecure coding practices can lead to serious vulnerabilities. By reviewing and analyzing the code, I improved my ability to spot common flaws like SQL injection, hardcoded credentials, and unsafe file handling. Overall, this task boosted my skills as a penetration tester and highlighted the importance of writing secure code from the start.

